



空间数据库原理与应用

第九讲：空间数据库索引与查询一

任课教师：李晓明

建筑与城市规划学院 城市空间信息工程系

其中部分图片来自互联网和同行专家

目录

CONTENTS

01

空间数据库索引

02

空间数据库查询

目录

CONTENTS

1

空间数据库索引

1.1 数据库物理存储

1.2 数据库索引

1.3 空间索引

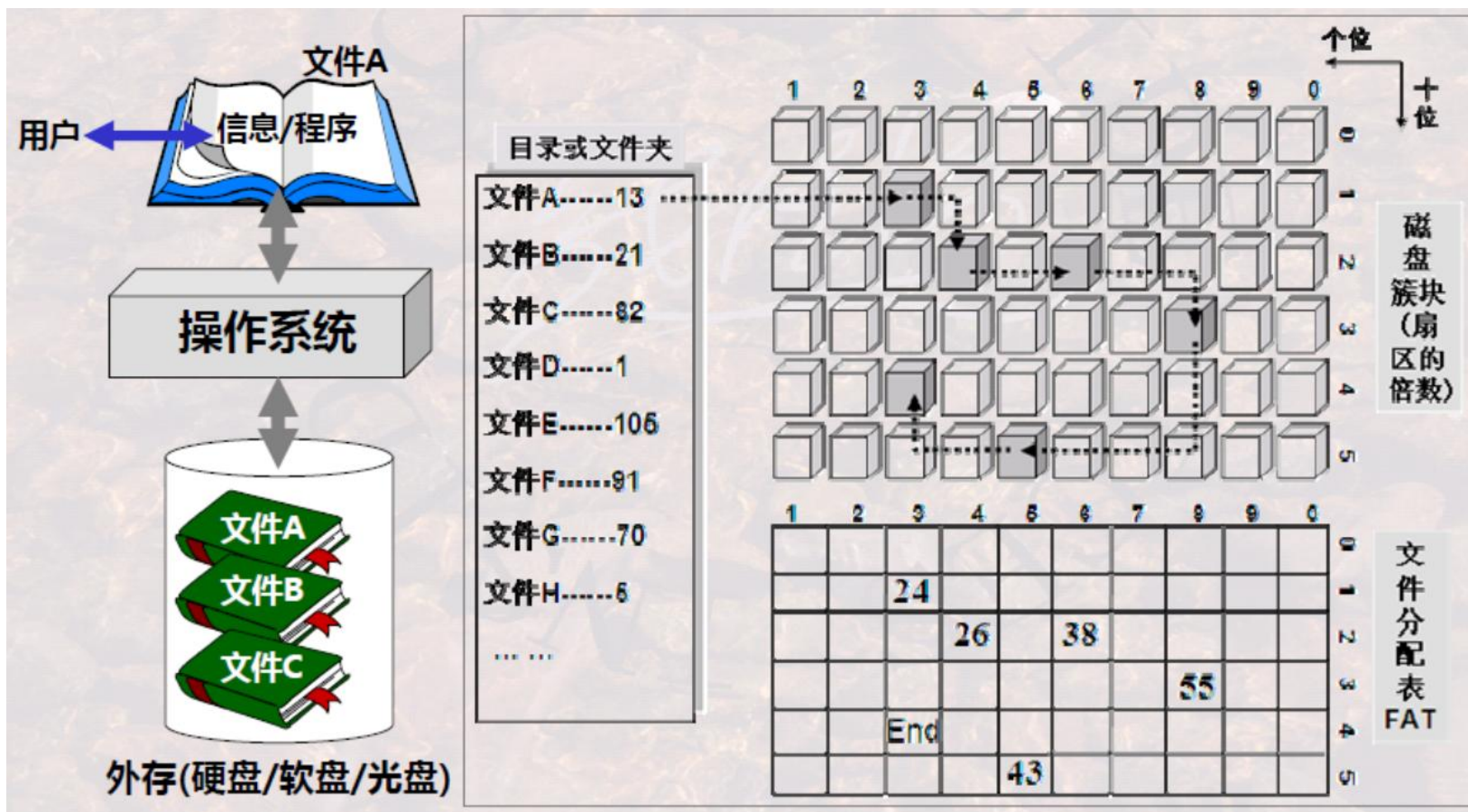
1.4 PostGIS的空间索引

1.1 数据库物理存储

计算机存储器结构体系

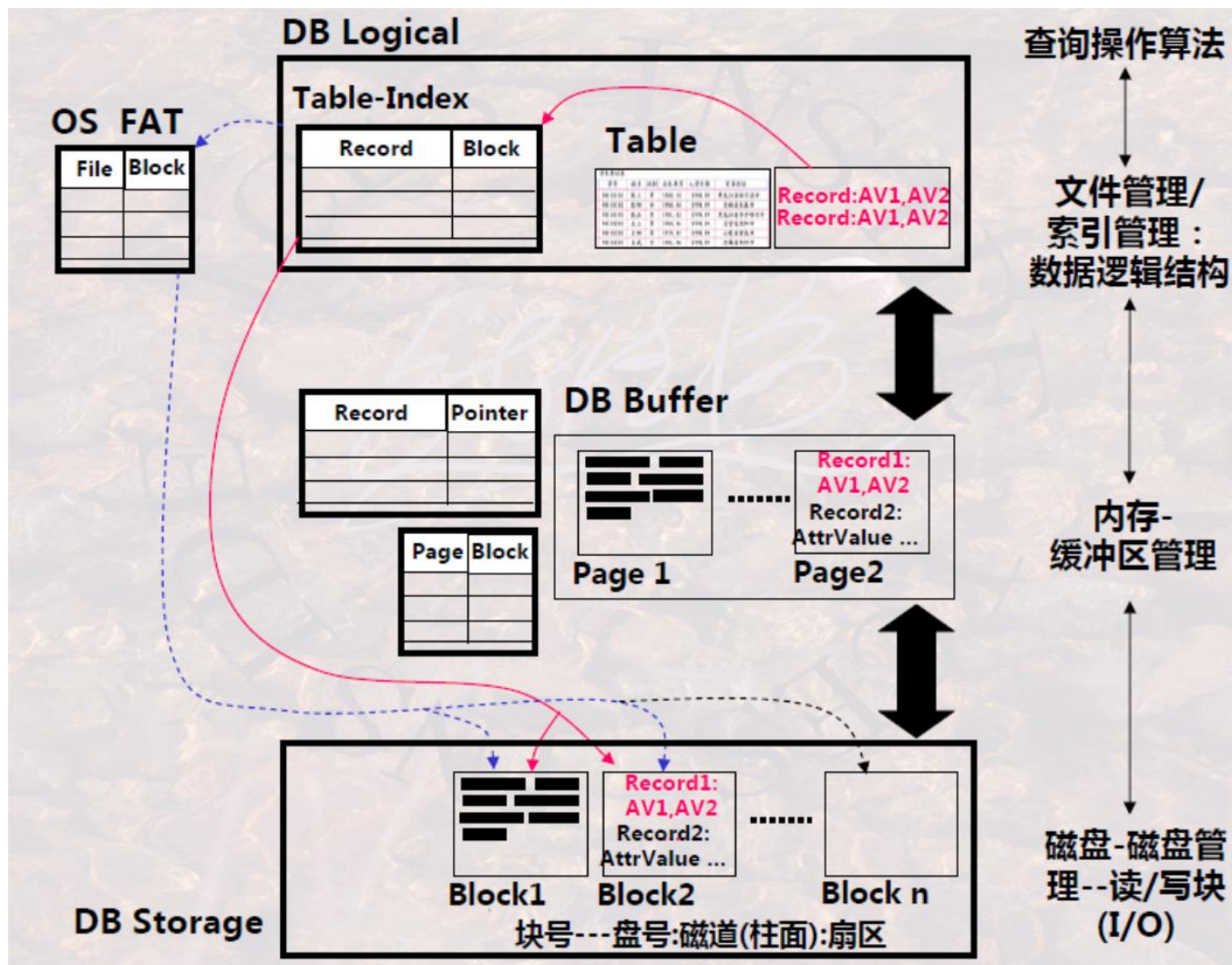
操作系统对数据的组织：FAT-目录(文件夹)-磁盘块/簇

FAT(文件分配表-File Allocation Table)



1.1 数据库物理存储

数据存储的映射关系



1.1 数据库物理存储

记录与磁盘块的映射

SQL—表:记录:属性值

学生登记表

学号	姓名	性别	出生年月	入学日期	家庭住址
98110101	张三	男	1980.10	1998.09	黑龙江省哈尔滨市
98110102	张四	女	1980.04	1998.09	吉林省长春市
98110103	张五	男	1981.02	1998.09	黑龙江省齐齐哈尔市
98110201	王三	男	1980.06	1998.09	辽宁省沈阳市
98110202	王四	男	1979.01	1998.09	山东省青岛市
98110203	王武	女	1981.06	1998.09	河南省郑州市

Storage—盘面:磁道:扇区

Block1

01010101010101010101	98110101	张三	男	1980.10	1998.09	黑龙江省哈尔滨市
0101001010101010091010	98110102	张四	女	1980.04	1998.09	吉林省长春市
1001100000101010101010	98110103	张五	男	1981.02	1998.09	黑龙江省齐齐哈尔市

Block5

01010101010101010101	98110101	张三	男	1980.10	1998.09	黑龙江省哈尔滨市
010100101010101010091010	98110102	张四	女	1980.04	1998.09	吉林省长春市
1001100000101010101010	98110103	张五	男	1981.02	1998.09	黑龙江省齐齐哈尔市

Block8

01010101010101010101	98110101	张三	男	1980.10	1998.09	黑龙江省哈尔滨市
010100101010101010091010	98110102	张四	女	1980.04	1998.09	吉林省长春市
1001100000101010101010	98110103	张五	男	1981.02	1998.09	黑龙江省齐齐哈尔市

逻辑
层面

表

记录:属性值

物理
层面

Storage Schema

文件

FAT

磁盘块

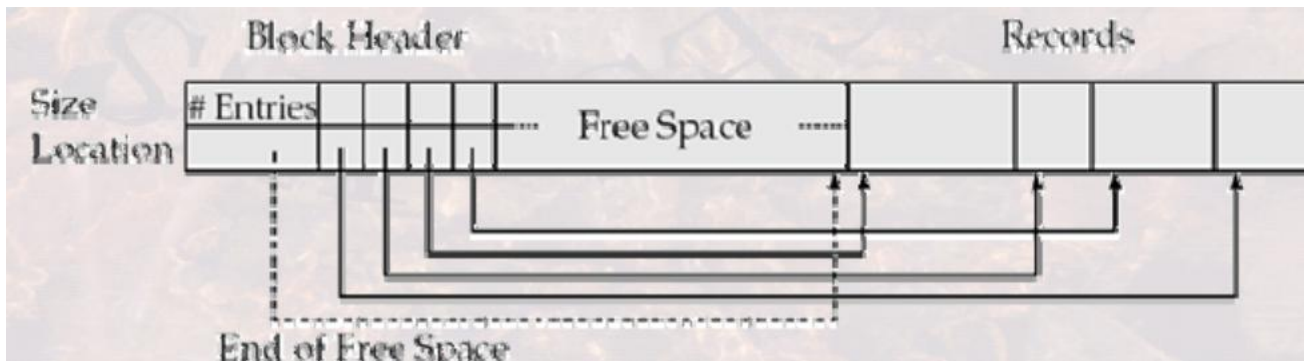
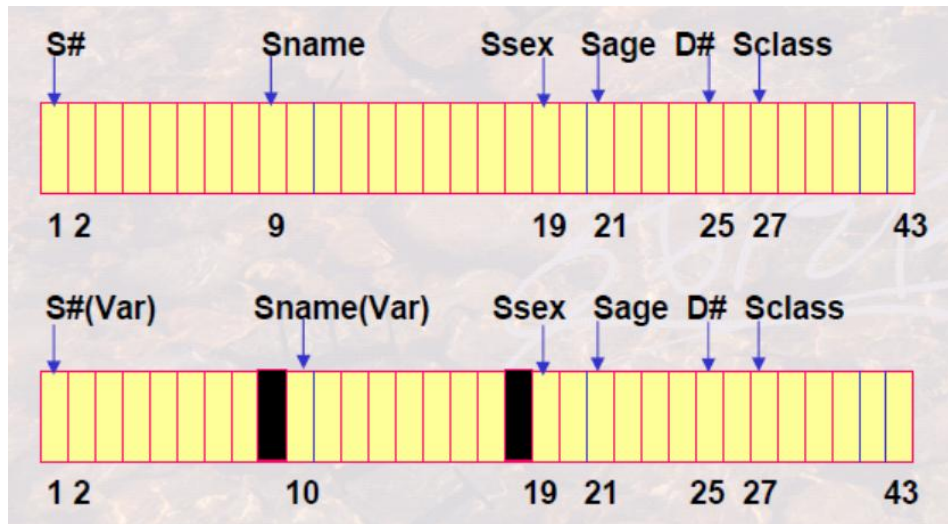
块内0/1数据

盘面:磁道:扇区

1.1 数据库物理存储

记录与磁盘块的映射

定长记录，还是变长记录(靠分隔符区分开始与结束)

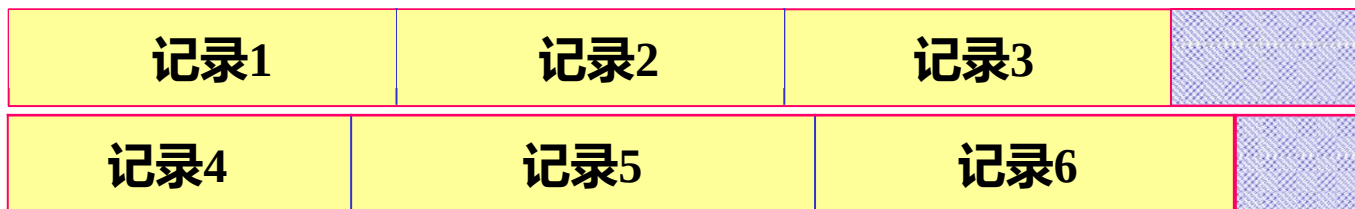


1.1 数据库物理存储

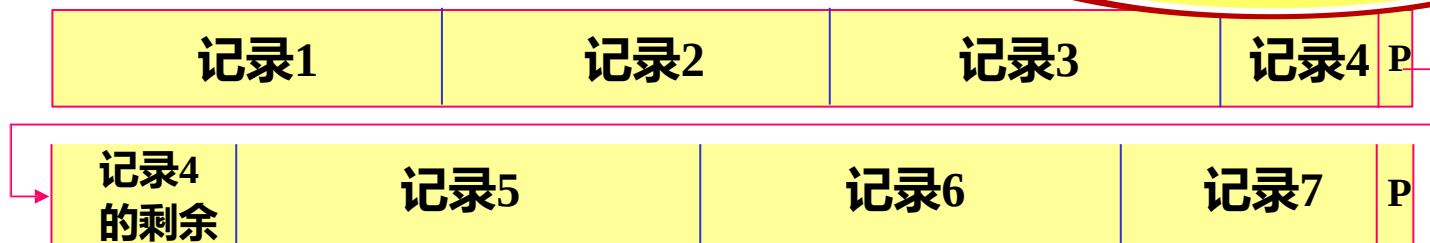
记录与磁盘块的映射

□记录是非跨块存储，还是跨块存储(靠指针连接)

- 浪费一些存储空间
- 磁盘块间无关联可并行



- 节省一些存储空间
- 磁盘块间有关联需串行



数据库-表所占磁盘块的分配方法

链接分配: 数据块中包含指向下一数据块的指针(访问速度问题)

按簇分配: 按簇分配, 簇是若干连续的磁盘块, 簇之间靠指针连接;

簇有时也称片段Segment或盘区extent

索引分配: 索引块中存放指向实际数据块的指针





Oracle 物理存储

数据库(database)	数据库(如SCT)																	
表空间(tablespace)	tspace1												SYSTEM					
文件(datafile)	fname1						fname2						fname3					
表(table)	student	dept	sc	course	teacher	Index1												
段(segment)	DATA	DATA	DATA	DATA	DATA	INDEX												
盘区(extent)																		
基本数据块(data block)																		



Oracle 物理存储

► 简要理解

- 每个数据库分成一个或多个表空间，所有表空间的组合存储容量即是数据库的存储容量
- 有系统表空间SYSTEM和用户表空间；系统表空间由Oracle在创建数据库时自动创建，用于数据字典等的管理；用户表空间可由用户创建
- 每个表空间由一个或多个操作系统文件构成，一个操作系统文件只能与一个数据库相联，操作系统文件仅起占位的作用。数据在表空间中可跨文件进行操作。
- 操作系统文件中存储一个表(Table)或多个表，一个表可存储在一个文件中

也可能存储在多个文件中。

- 上述为逻辑存储层

database tablespace data file table	数据库(如SCT)														
	tspace1										SYSTEM				
	fname1					fname2					fname3				
	student	dept	sc	course	teacher	Index1									
	DATA	DATA	DATA	DATA	DATA	INDEX									

1.1 数据库物理存储

战德臣教授

Oracle 物理存储

➤ 简要理解

一个表空间
可以由多个
文件构成

表空间的文件
是否可自
动扩展

```
CREATE TABLESPACE tblspname
  DATAFILE 'filename'
    [SIZE n [K|M]]
    [REUSE]
    [AUTOEXTEND OFF | AUTOEXTEND ON
      [NEXT n [K|M] [MAXSIZE {UNLIMITED | n [K|M]}]
      {, 'filename' ...}]
    [ONLINE | OFFLINE]
    [DEFAULT STORAGE ([ INITIAL n] [NEXT n] [MINEXTENTS n]
      [MAXEXTENTS {n | UNLIMITED}] [PCTINCREASE n])]
    [MINIMUM EXTENT n [K|M]] ;
```

数据库(如SCT)											
tspace1						SYSTEM					
fname1			fname2			fname3					
student	dept	sc	course	teacher	Index1						
DATA	DATA	DATA	DATA	DATA	INDEX						

可设置表空间的初始
容量、最小盘区数、
每次扩展容量等

盘区的大小
是可以动态
调整的

可设置构成
表空间的最
小盘区

1.2 数据库索引

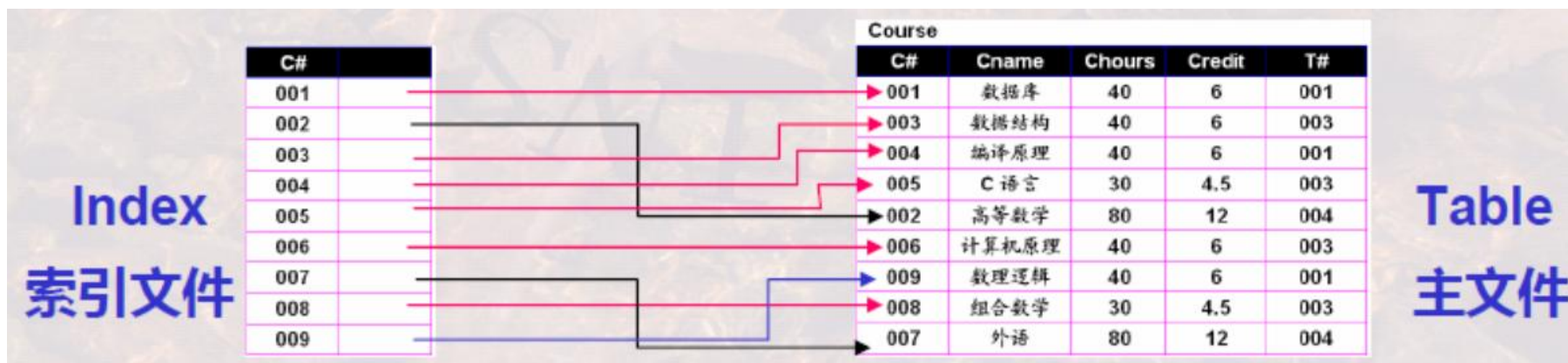
定义

索引是定义在存储表(Table)基础之上, 有助于无需检查所有记录而快速定位所需记录的一种**辅助存储结构**, 由一系列存储在磁盘上的**索引项**(index•entries)组成, 每一索引项又由两部分构成:•

索引字段: 由Table中某些列(**通常是一列**)中的值串接而成。索引中**通常**存储了索引字段的每一个值(也有不是这样的)。索引字段类似于词典中的**词条**。

行指针: 指向Table中包含索引字段值的记录在磁盘上的存储位置。行指针类似于词条在书籍、词典中出现的**页码**。

存储索引项的文件为**索引文件**, 相对应, 存储表又称为**主文件**



1.2 数据库索引

定义

索引文件是一种辅助存储结构，其**存在与否不改变存储表的物理存储结构**；然而其存在，可以明显提高存储表的访问速度。

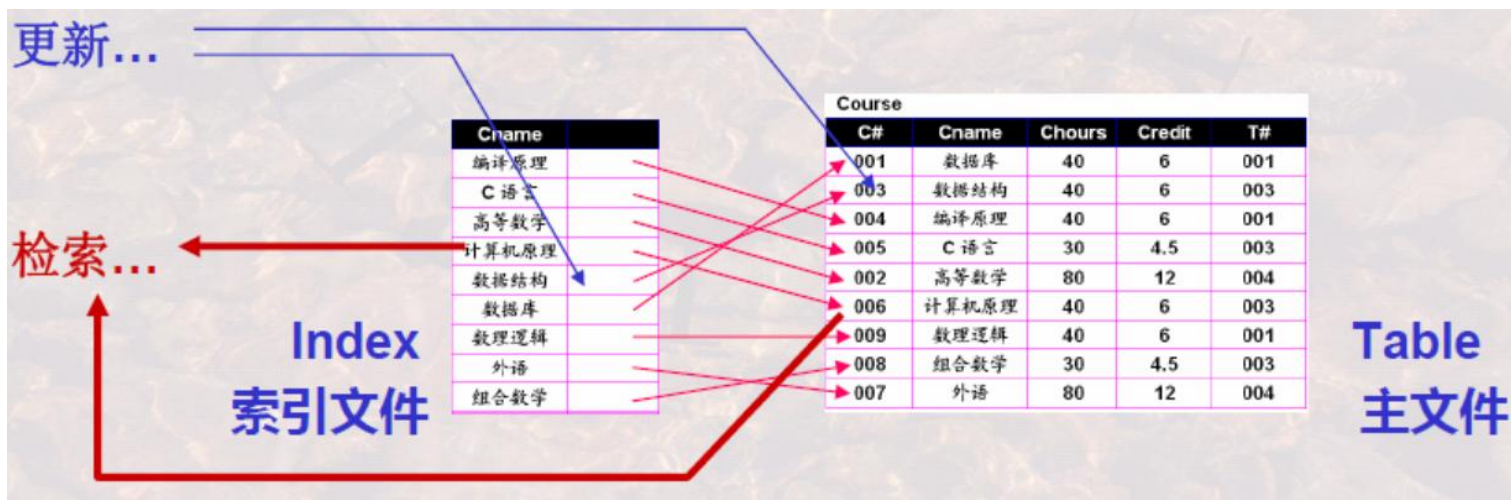
索引文件组织方式有两种：（相对照的，主文件组织有堆文件、排序文件、散列文件、聚簇文件等多种方式）

排序索引文件(Ordered•indices):•按索引字段值的某一种顺序组织存储

散列索引文件(Hash•indices):•依据索引字段值使用散列函数分配散列桶的方式存储

索引文件比主文件**小很多**。通过检索一个**小的索引文件(可全部装载进内存)**，快速定位后，再有针对性的读取**非常大的主文件**中的有关记录

有索引时，更新操作必须同步更新索引文件和主文件。



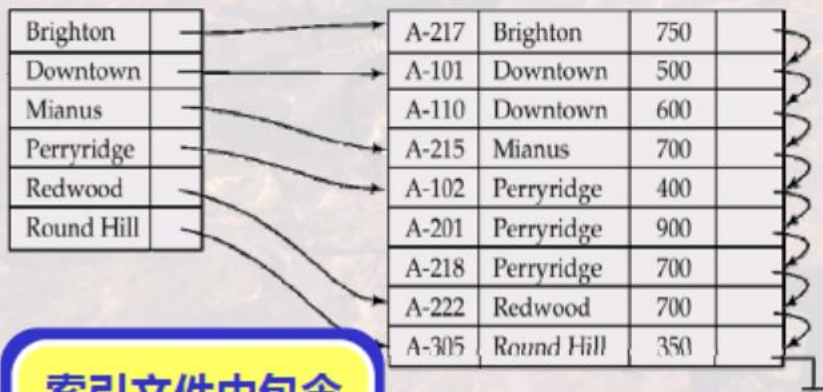
1.2 数据库索引

稠密索引与稀疏索引

对于主文件中每一个记录(形成的每一个索引字段值), 都有一个索引项和它对应, 指明该记录所在位置。这样的索引称**稠密索引(dense index)**

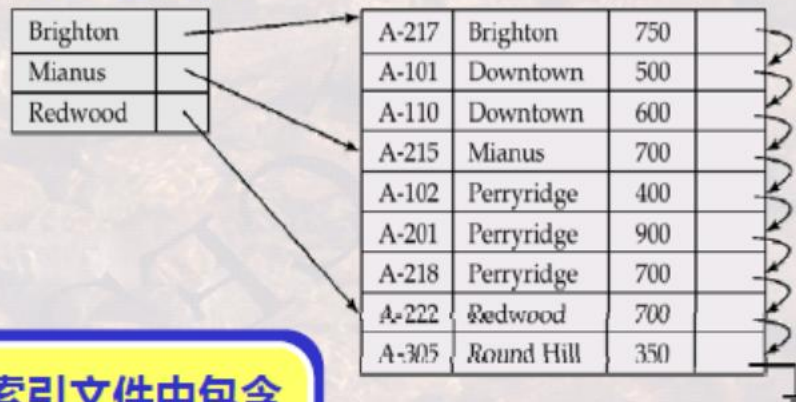
对于主文件中部分记录(形成的索引字段值), 有索引项和它对应, 这样的索引称**非稠密索引(undense index)**或**稀疏索引(sparse index)**

稠密索引



索引文件中包含了主文件对应字段的**所有**不同值

稀疏索引



索引文件中包含了主文件对应字段的**部分**不同值

1.2 数据库索引

主索引

主索引 通常是对每一存储块有一个索引项，索引项的总数和存储表所占 的存储块数
目相同，存储表的每一存储块的第一条记录，又称为锚记录 (anchor record), 或简称为
块锚(block anchor)

- 主索引的索引字段值为块锚的索引字段值，而指针指向其所在的存储块。
- 主索引是按索引字段值进行排序的一个有序文件，通常建立在有序主文件 的基于主码的
排序字段上，即主索引的索引字段与主文件的排序码(主码)有对 应关系

主文件

- 主索引是稀疏索引。

主索引文件

C#	
001	
004	
007	

块锚主码值

块指针

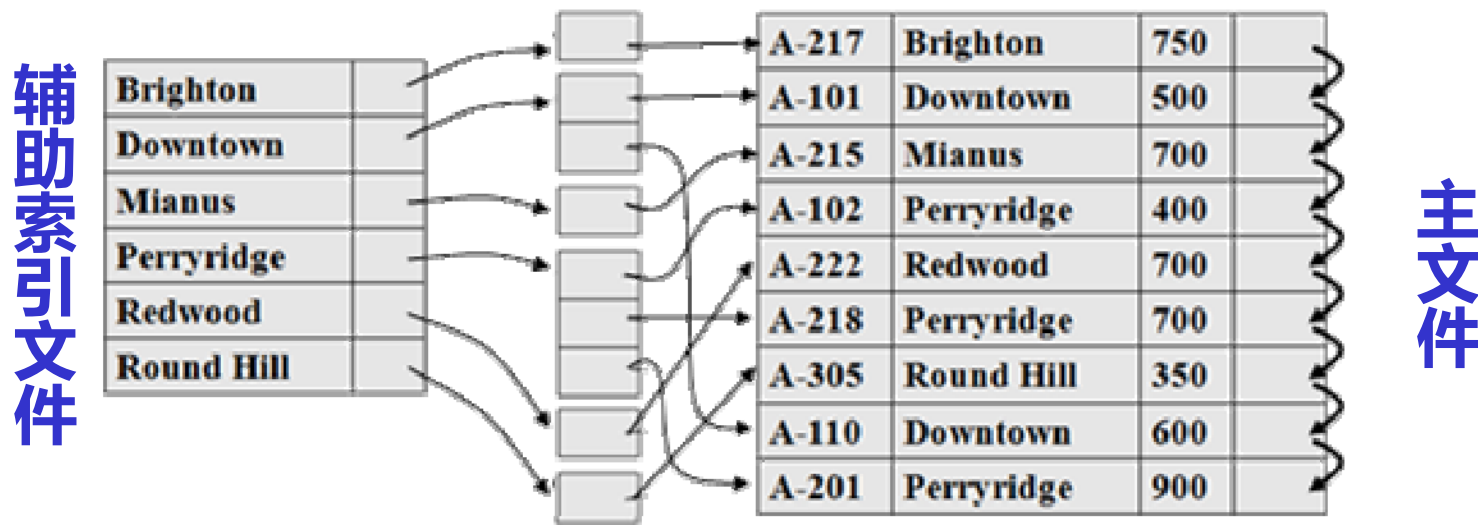
C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
002	高等数学	80	12	004
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
006	计算机原理	40	6	003
007	外语	80	12	004
008	组合数学	30	4.5	003

1.2 数据库索引

辅助索引

辅助索引 是定义在主文件的任一或多个非排序字段上的辅助存储结构。

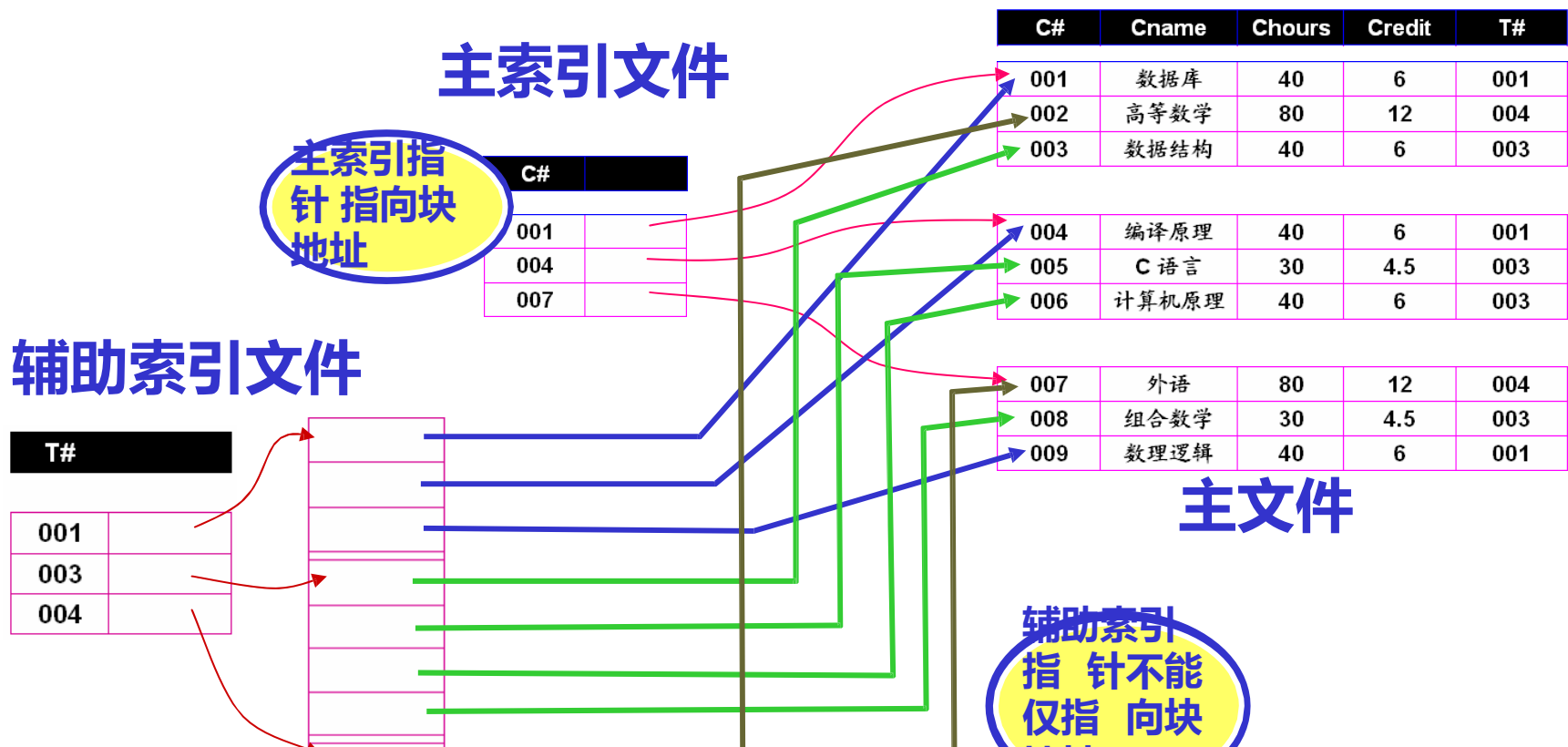
- 辅助索引通常是对某一非排序字段上的每一个不同值有一个索引项：索引 字段即是该字段的值，而指针则指向包含该记录的块或该记录本身；
- 当非排序字段为索引字段时，如该字段值不唯一，则要采用一个类似链表 的结构来保存包含该字段值的所有记录的位置。
- 辅助索引是稠密索引，其检索效率有时相当高。



1.2 数据库索引

主索引VS 辅助索引

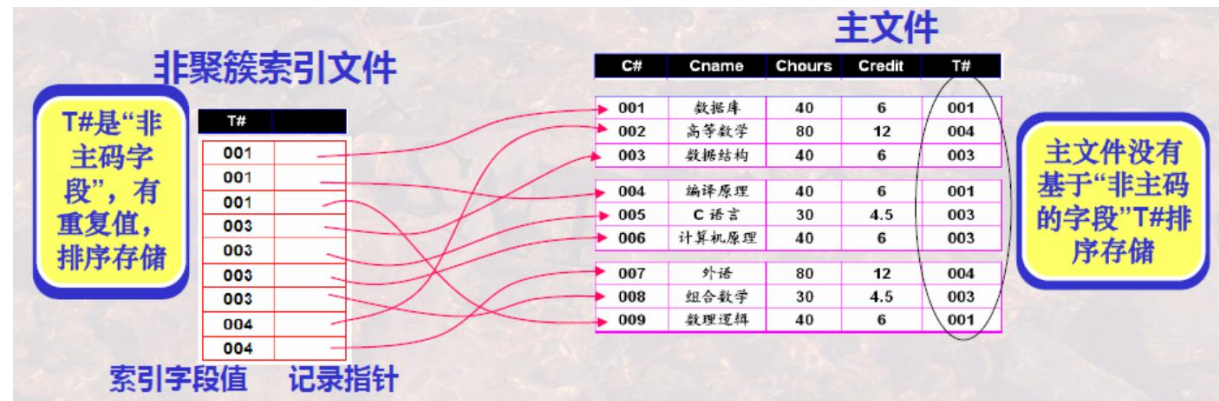
- 一个主文件仅可以有一个主索引，但可以有多个辅助索引
- 主索引通常建立于主码/排序码上面；辅助索引建立于其他属性上面
- 可以利用主索引重新组织主文件数据，但辅助索引不能改变主文件数据
- 主索引是稀疏索引，辅助索引是稠密索引



1.2 数据库索引

聚簇索引和非聚簇索引

聚簇索引——是指索引中邻近的记录在主文件中也是临近存储的；
非聚簇索引——是指索引中邻近的记录在主文件中不一定是邻近存储的。



□如果主文件的某一**排序**字段**不是主码**，则该字段上每个记录取值便不唯一，此时该字段被称为**聚簇字段**；
聚簇索引通常是定义在聚簇字段上。

□一个主文件只能有一个聚簇索引文件，但可以有多个非聚簇索引文件

□主索引通常是聚簇索引(但其索引项总数不一定和主文件中聚簇字段上不同值的数目相同，其和主文件存储块数目相同)；辅助索引通常是非聚簇索引。

□主索引/聚簇索引是能够决定记录存储位置的索引；而非聚簇索引则只能用于查询，指出已存储记录的位置。

1.2 数据库索引

B+树(B+ Tree)

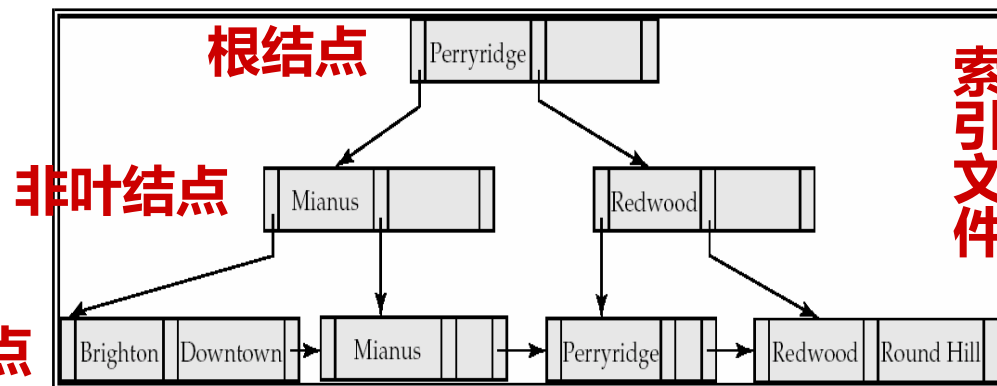
B+树索引：一种以树型数据结构来组织索引项的多级索引

所有记录节点都是按键值的大小顺序存放在同一层的叶子节点上，由各叶子节点指针进行连接。



一块中索引项的组织

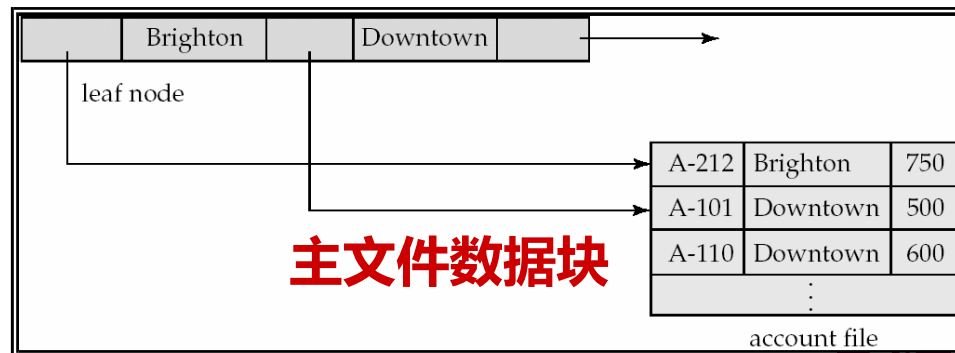
- K_i —索引字段值
- P_j —指针，指向索引块或数据块或数据块中记录的指针



索引文件的叶子结点

示例 存储块 = 4096 Byte
 整数型索引字段值 = 4 Byte
 指针 = 8 Byte
 则：n应满足 $4(n-1)+8n \leq 4096$
 n取最大值
n=341

•能够自动保持与主文件大小相 适应的树的层次

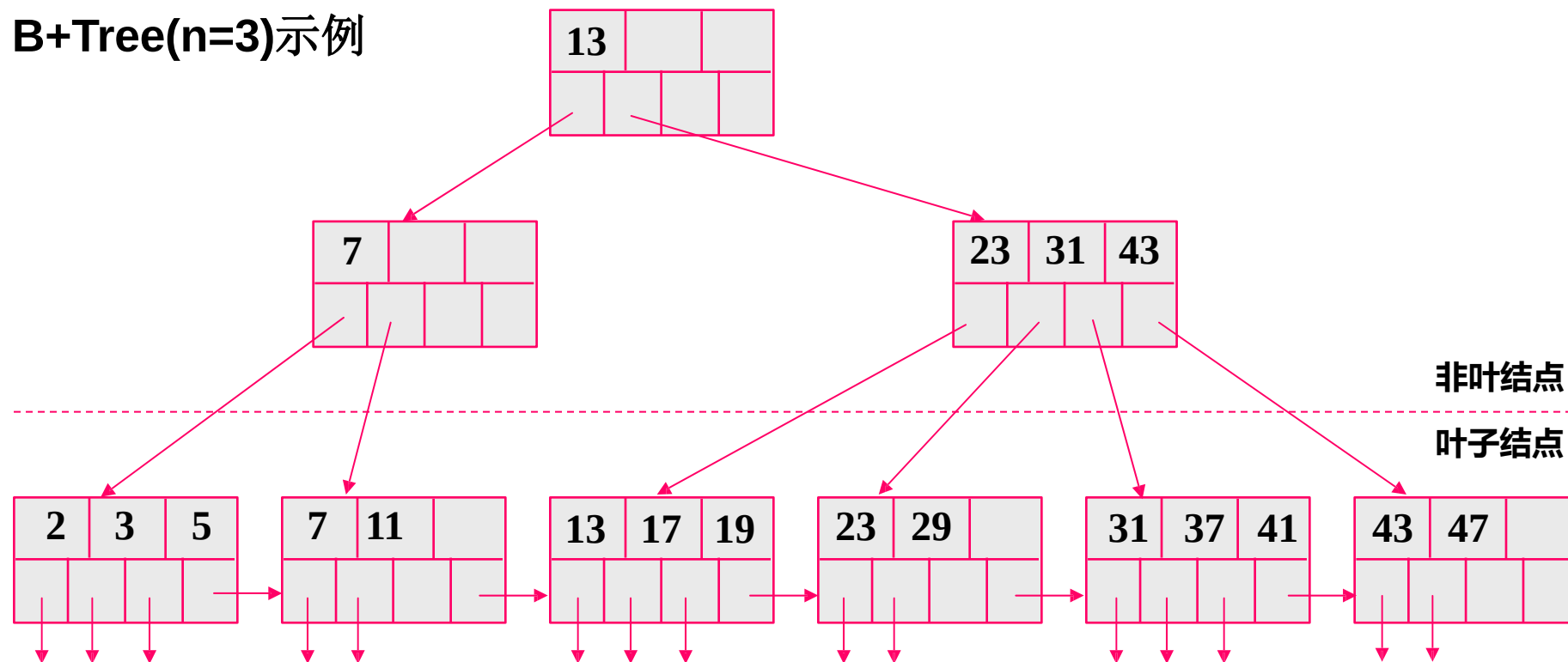


1.2 数据库索引

B+树(B+ Tree)

- 索引字段值重复出现于叶结点和非叶结点;

B+Tree(n=3)示例



- 指向主文件的指针仅出现于叶结点;
- 所有叶结点即可覆盖所有键值的索引;
- 索引字段值在叶结点中是按顺序排列的;

仅叶结点块的集合就是主文件完整的索引, 即: 最低一级(或一层)索引

1.2 数据库索引

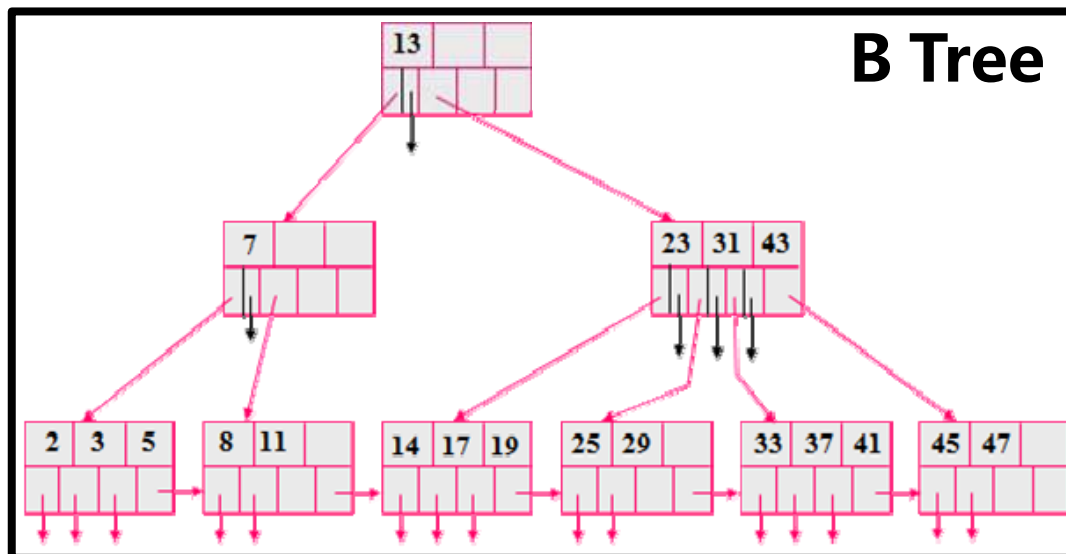
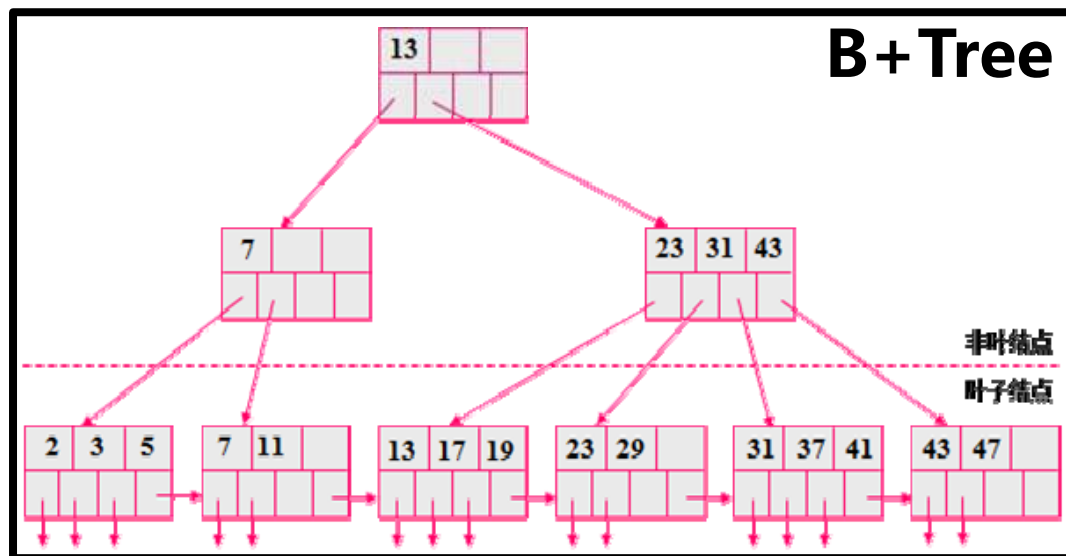
B+树 vs. B树

B Tree vs. B+ Tree

- 一块中存放的索引项个数是否相同?
- 索引字段值都出现在哪里?
- 指向主文件的指针存在哪里?
- 分裂与合并的方法是否一致?

B Tree

- 索引字段值仅出现一次或者在叶结点或者在非叶结点;
- 指向主文件的指针出现于叶结点或非叶结点;
- 所有结点才能覆盖所有键值的索引。



1.2 数据库索引

SQL语言中的索引创建与维护

SQL语言关于索引的基本知识

- 当定义Table后，如果定义了主键，则系统将自动创建主索引，利用主索引对Table进行快速定位、检索与更新操作;
- 索引可以由用户创建，也可以由用户撤消
- 当索引被创建后，无论是主索引，还是用户创建的索引，DBMS都将自动维护所有的索引，使其与Table保持一致，即：当一条记录被插入到Table中后，所有索引也自动的被更新
- 当Table被删除后(drop table), 定义在该Table上的**所有**索引将自动被撤消

1.2 数据库索引

SQL语言中的索引创建与维护

- 索引可以由用户在任何属性上创建

X/OPEN SQL中关于索引的语句

- 创建索引:

```
CREATE [unique] INDEX indexname  
ON tablename ( colname [asc | desc]  
               {, colname [asc | desc] . . . } );
```

- 示例：在student表中创建一个基于Sname的索引

```
create index idxSname on student(sname);
```

- 示例：在student表中创建一个基于Sname和Sclass的索引

```
create index idxSnamcl on student(sname, sclass);
```

- 如某索引不再需要，则可通过撤消命令，撤消用户创建的索引

```
DROP INDEX indexname;
```

1.2 数据库索引

SQL语言中的索引创建与维护

- 选择哪些属性创建索引，以及如何创建与维护索引，如何利用索引改善数据库的运行性能，是DBA(数据库管理员)的重要职责。
- 是否建立和在哪些属性上建立索引需要考虑：访问时间、插入时间、删除时间与空间负载。既要改善性能，又要控制代价。
- 建立索引还需考虑索引的类型：索引如何支持存取的有效性，比如：支持的是属性的限定值(是否符合单一值)，还是支持属性的限定范围的值(是否符合一定范围)

对哪些属性
建立索引?

对经常出现在检索条件、
连接条件、分组计算条件
中的属性可建立索引

SELECT...FROM...WHERE...GROUP BY...

1.3 空间索引

空间索引概念

空间索引:

空间索引就是指依据空间对象的位置和形状或空间对象之间的某种空间关系，按一定的顺序排列的一种数据结构，其中包含空间对象的概要信息，如对象的标识、外接矩形及指向空间对象实体的指针。

1.3 空间索引概述

问题的提出

- 索引的意义
 - 空间索引，也称为空间访问方法（Spatial access method-SAM）
 - 就是指依据空间对象的位置和形状或者空间对象之间的某种空间关系按一定的顺序排列的一种数据结构。
 - 包含空间对象的概要信息，如对象的标识、外包络矩形、及指向空间对象实体的指针。
 - 空间数据库索引技术是对存储在介质上的数据位置信息的描述，是为了快速访问一条特定查询所请求的数据，而无需遍历整个数据库。

1.3 空间索引

常见的空间索引

对象范围索引

格网索引

四叉树索引

BSP树索引

R树和R+树索引

1.3 R树索引

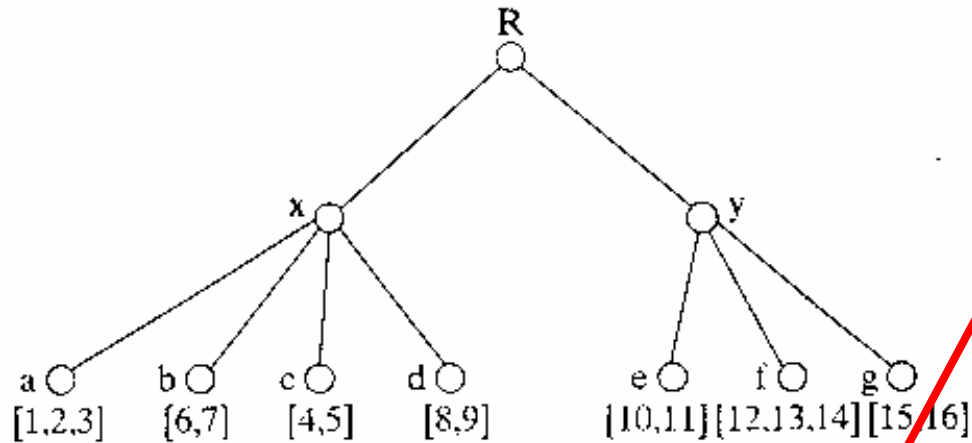
R树

- R树是处理空间数据的B +树的改进，它像B +树一样，是一个高度平衡的数据结构。
- R 树算法是由美国加州大学的Antomm Guttman 教授在1984 年提出的一种空间数据库动态索引算法，现已成为空间数据库索引中应用最广泛的算法之一。
- R 树较好地解决了许多传统算法未解决的空间数据动态索引问题。

1.3 R树索引

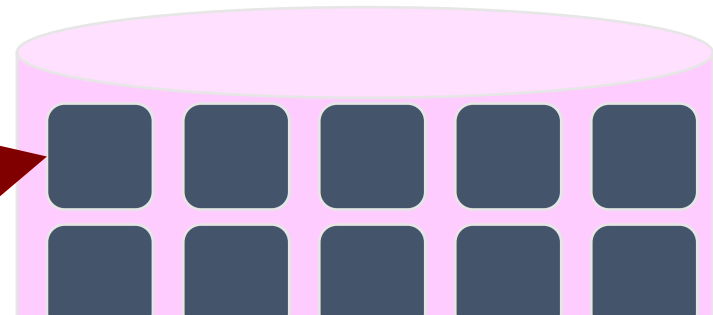
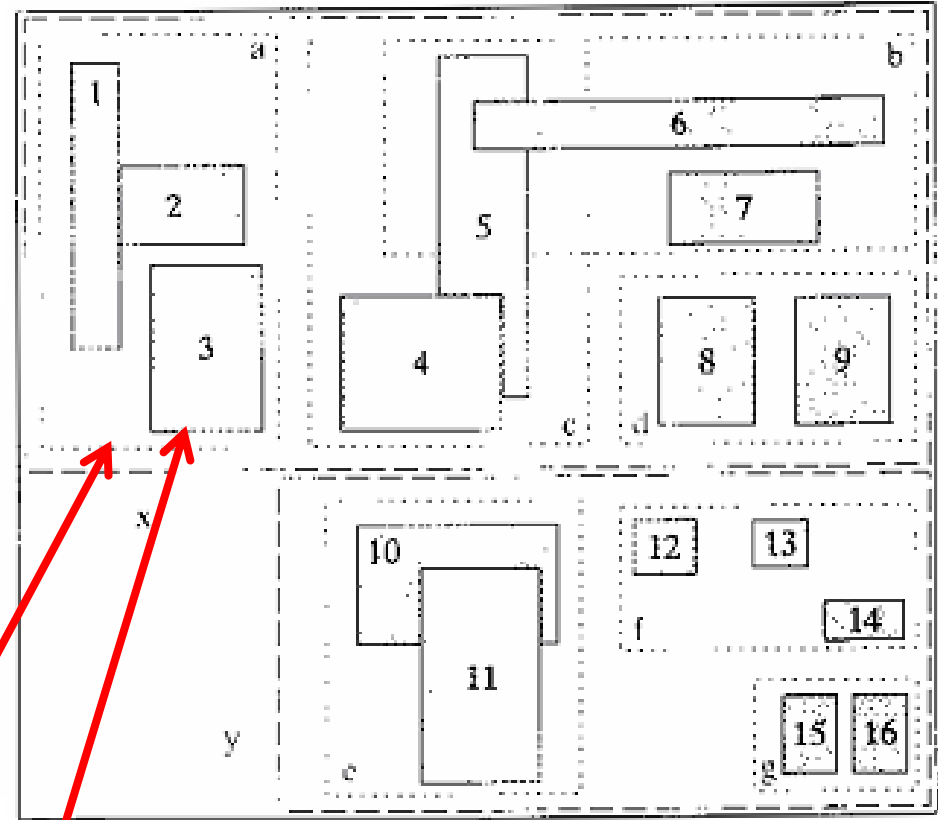
R树

MBR空间范围可以相交，但所有对象仅属于一个MBR



ID1	x1	y1	x2	y2	ID2

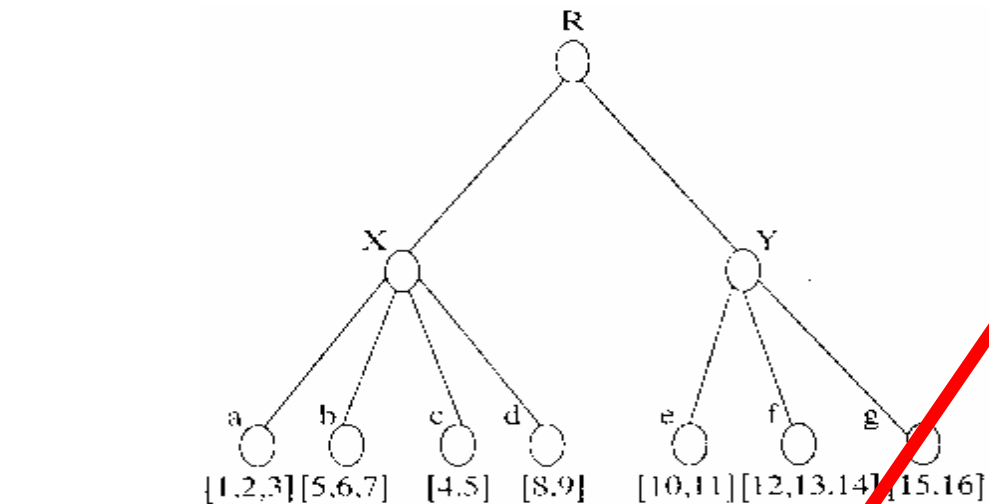
ID2	x1	y1	x2	y2	P



1.3 R树索引

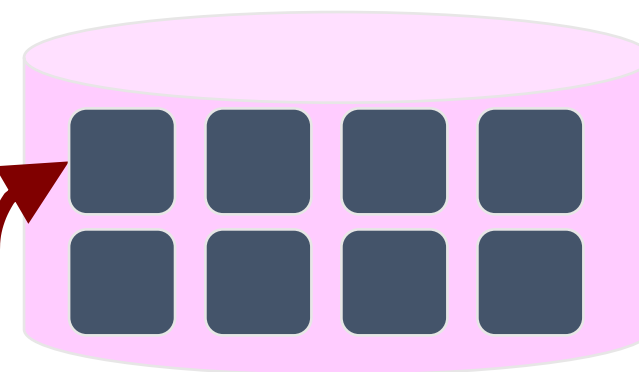
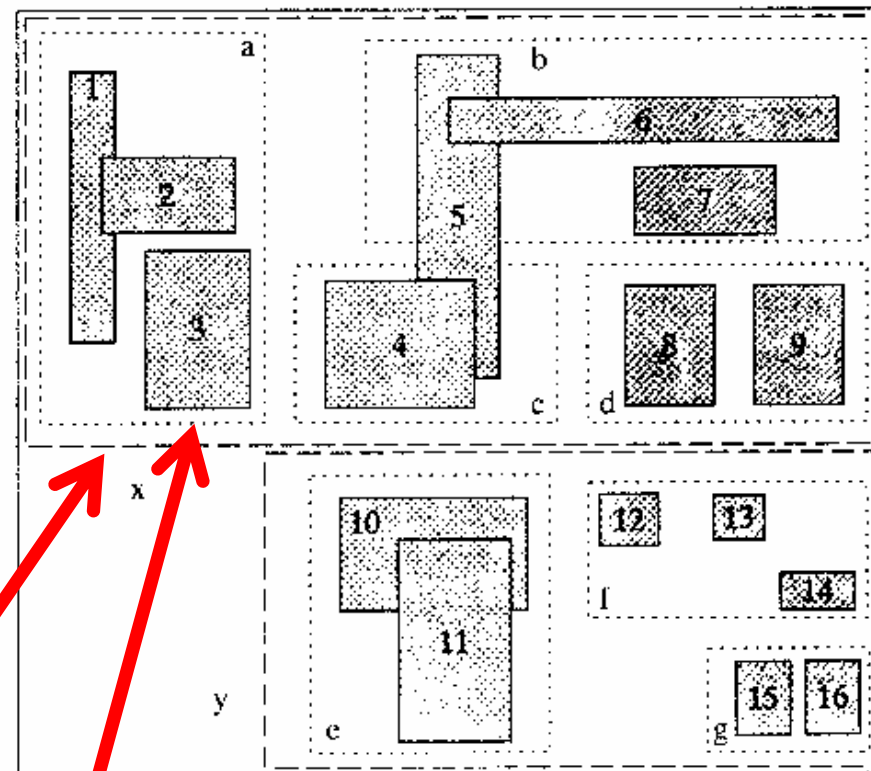
R+树

所有MBR不相交，但对象可以属于多个MBR



ID1	x1	y1	x2	y2	ID2

ID2	x1	y1	x2	y2	P



1.4 PostgreSQL的索引

postgresql提供的索引类型有：B-tree、hash、gist和gin。大多情况下，B-tree索引比较常用，用户可以使用create index命令创建一个B-tree索引。

1、B-tree索引：

B-tree适合处理那些能够按顺序存储的数据，比如对于一些字段涉及使用：< ,<= ,= ,>= 或 >操作符之一进行比较的时候，可以建立一个索引。

也可以使用B-tree索引搜索来实现与这些运算符的组合相同的构造，如BETWEEN和IN。此外，索引列上的IS NULL或IS NOT NULL条件可以与B-tree索引一起使用。

2、hash索引：

hash索引只能处理简单的等于比较。当一个索引的列涉及使用=操作符进行比较的时候，查询规划器会考虑使用hash索引。

3、gist索引：

gist索引不是单独一种索引类型，而是一种架构，可以在这种架构上实现很多不同的索引策略。因此，可以使用gist索引的特定操作符类型高度依赖于索引策略（操作符类）

4、GIN索引

GIN索引是反转索引，可以处理包含多个键的值（比如数组）。与gist类似，gin支持用户定义的索引策略，可以使用GIN索引的特定操作符类型根据索引策略的不同而不同。

1.4 PostGIS空间索引

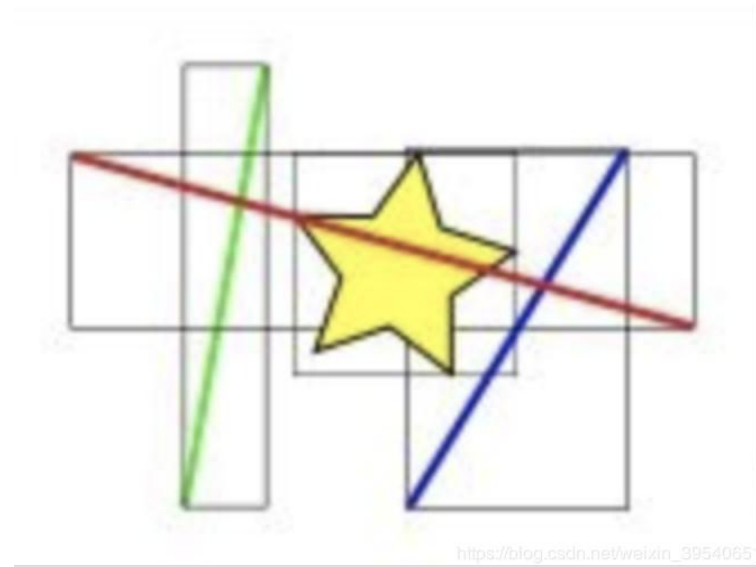
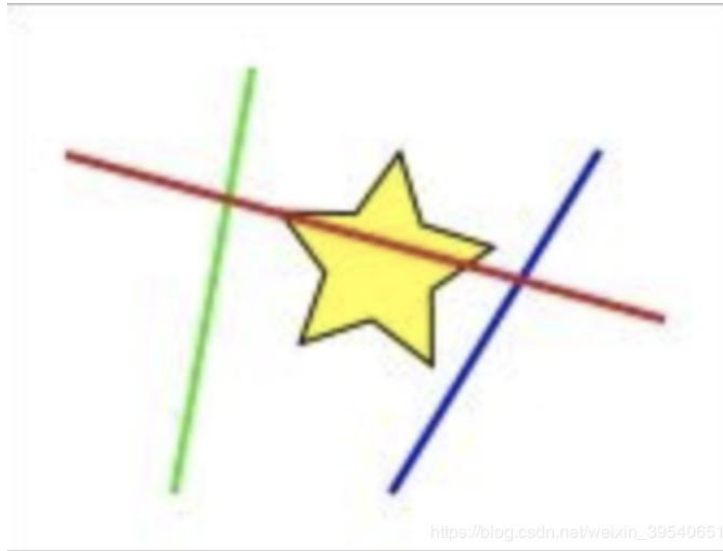
空间索引是postgis中十分重要的功能，一个数据库中如果不支持索引那几乎是没法使用的。postgis中空间索引通过将数据组织到搜索树中来加快搜索速度，搜索树可以快速遍历以查找特定记录。

对于空间的几何图形，PostGIS不是通过btree索引来加速查询，而是通过gist索引。gist索引是通过R_tree的结构来实现对空间类型数据的索引查询，其结构类似于btree索引。

1.4 PostGIS空间索引

gist索引工作原理

查询和黄星相交的对象，即图中的红线， gist索引不能索引几何要素本身，而是索引几何要素的边界框 (bouding box) 。



在索引计算时，其实是先判断那些边界框和黄星所在的框相交，显然是红线和蓝线的框，然后再进行哪些直线与黄星相交的精确计算。

1.4 PostGIS空间索引

gist索引工作原理

对一张数据量1000的表进行全表扫描:

```
postgis=# select count(*) from t_pos;
```

```
count
```

```
-----
```

```
1000
```

```
(1 row)
```

```
postgis=# explain (analyze,buffers,timing) select * from t_pos where pos && st_astext(' POINT(73.689759 3.880185)');
```

```
QUERY PLAN
```

```
-----
```

```
Seq Scan on t_pos (cost=0.00..21.50 rows=1 width=36) (actual time=0.012..0.600 rows=1 loops=1)
```

```
Filter: (pos && '0101000000C9C7EE02256C52402670EB6E9E0A0F40'::geometry)
```

```
Rows Removed by Filter: 999
```

```
Buffers: shared hit=9
```

```
Planning Time: 0.231 ms
```

```
Execution Time: 0.612 ms
```

```
(6 rows)
```

1.4 PostGIS空间索引

gist索引工作原理

使用语法:

```
postgis=# create index idx_t_pos_1 on t_pos using gist(pos);  
CREATE INDEX
```

加上索引，使用索引查询:

```
postgis=# explain (analyze,buffers,timing) select * from t_pos where pos && st_astext(' POINT(73.689759 3.880185)');  
QUERY PLAN
```

```
-----  
Index Scan using idx_t_pos_1 on t_pos (cost=0.14..8.16 rows=1 width=36) (actual time=0.057..0.058 rows=1 loops=1)
```

```
  Index Cond: (pos && '0101000000C9C7EE02256C52402670EB6E9E0A0F40'::geometry)
```

```
  Buffers: shared hit=3
```

```
Planning Time: 0.228 ms
```

```
Execution Time: 0.074 ms
```

```
(5 rows)
```

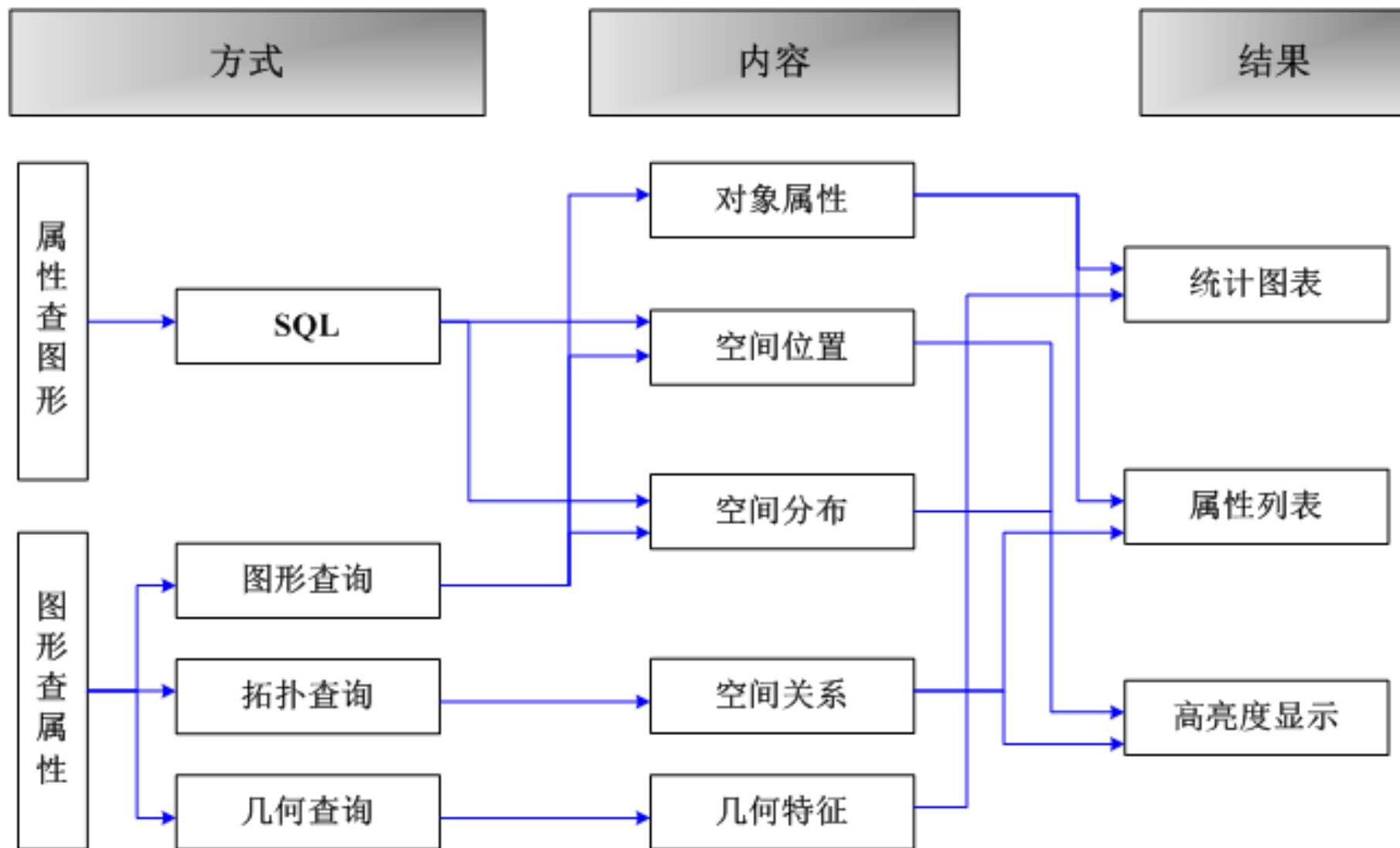
目录

CONTENTS

2

空间数据库查询

空间查询方式



空间数据查询的方式、内容与结果

■ 标准的SQL查询语言

SQL是关系数据库管理系统（RDBMS）的标准查询语言，是一种声明性语言，即用户只需描述所要的结果即可，而不必描述获得结果的过程。

SQL语句由命令、从句、运算符、合计函数等构成。SQL的功能包括：查询、操作、定义、控制，是一个综合的、通用的、功能极强的关系数据库语言。

SELECT语句一般格式：

SELECT[ALL|DISTINCT]<目标类表达式>[别名]...

FROM<表名或视图名>[别名][,<表名或视图名>[别名]].....

[**WHERE**<条件表达式>]

[**GROUP BY** <列名1>[**HAVING**<条件表达式>]]

[**ORDER BY** <别名2>[ASC|DESC]];

■ 标准的SQL查询语言

• SQL语言的不足

- 1) 数据类型太简单，只有整型、日期型、字符串型等，不能支持点线面等复杂数据类型（没有相应的操作原语）。
- 2) 所有数据类型都是固定的，无法增加新的数据类型。

• 解决方法：

- 1) 利用BLOB类型字段存储空间信息，依赖于宿主语言的应用程序代码。
- 2) 混合系统，空间数据以文件形式保存在数据库外部（比如，MapInfo）。无法利用数据库的服务，比如查询语言、并发控制、索引支持等。
- 3) 利用OR-DBMS自定义ADT。比如，自定义polygon数据类型，以及相应的函数方法，比如adjacent。

■ 扩展的SQL查询语言

扩展的SQL查询语言主要类型

查询谓词的扩展

- 增加空间数据类型 （点、线、面）
- 增加空间操作算子 （长度、面积）
- 增加查询条件 （临近、叠加、经过）

■ 扩展的SQL查询语言

OGIS的SQL空间扩展

- 支持四个子空间几何类型：
 - Point; Curve; Surface; GeometryCollection

- 操作分类：

- 1) 用于所有几何类型的基本操作。

例如，SpatialReference返回几何体采用的坐标系统

- 2) 用于空间对象之间拓扑关系的操作测试。

例如，overlap返回两个对象是否相交。

- 3) 用与空间分析的一般操作。

例如，distance返回两个空间对象之间的最短距离。

扩展的SQL查询语言

OGIS的SQL空间扩展

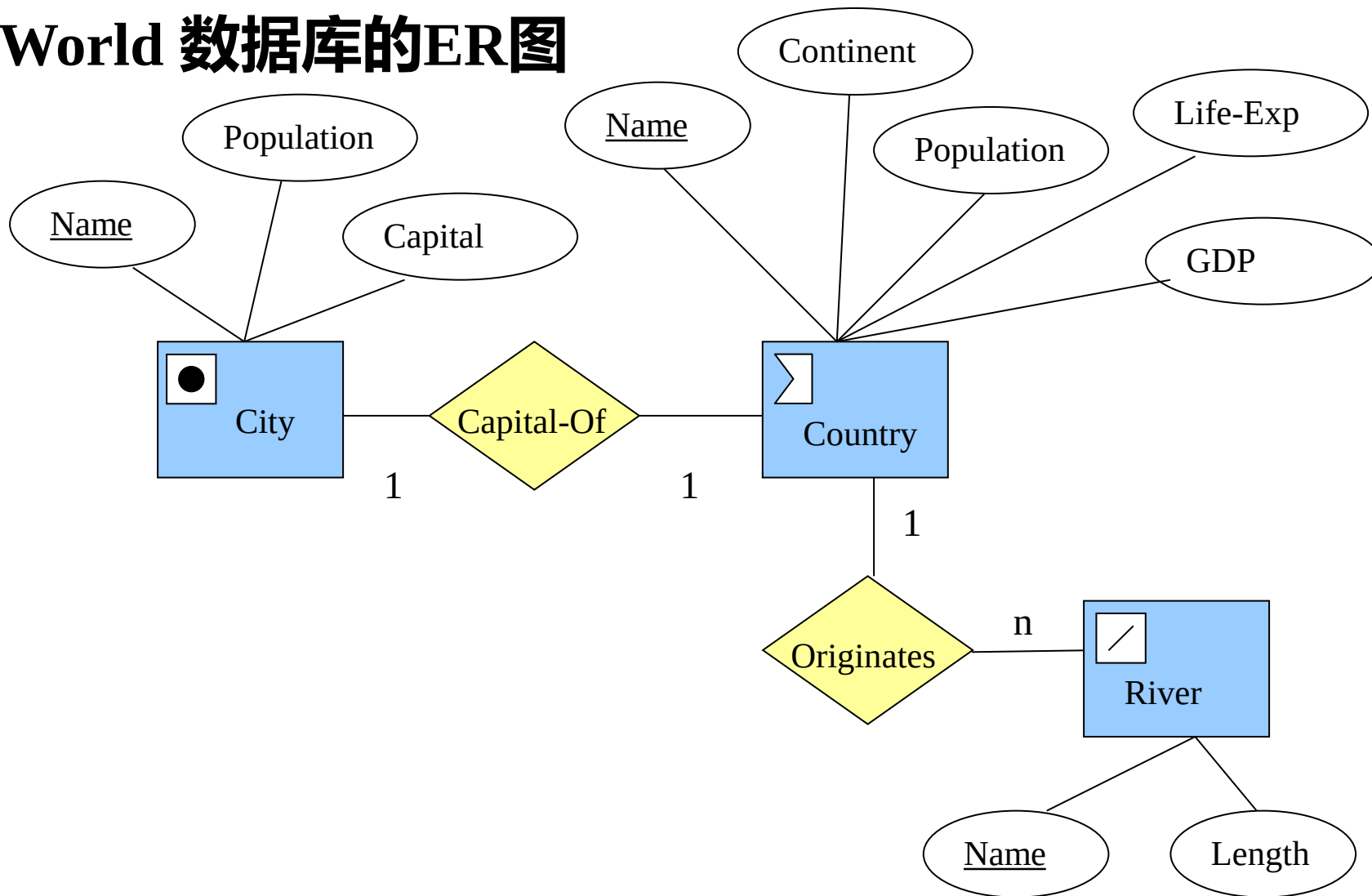
• 基本操作

基本函数	SpatialReference	几何体的坐标系
	Envelope	几何体最小外接矩形
	Export	其他形式的几何体
	IsEmpty	判断几何体是否为空
	IsSimple	判断几何体是否自交
	Boundary	几何体的边界
拓扑/集合运算	Equal	判断两个几何体是否相等
	DisJoint	判断几何体是否相接
	Intersect	判断几何体是否相交
	Touch	判断几何体是否相邻
	Cross	判断线与面是否相交
	Within	判断是否被包含
	Contains	判断是否包含
	Overlap	判断是否有重叠
空间分析	Distance	几何体之间最短距离
	Buffer	几何体缓冲区
	ConvexHull	几何体最小凸包
	Intersection	两个几何体的交集
	Union	两个几何体的并集
	Difference	几何体的差集
	SymmDiff	两个几何体的不相交部分

扩展的SQL查询语言

OGIS的SQL空间扩展

实例：World 数据库的ER图



扩展的SQL查询语言

OGIS的SQL空间扩展

- World数据库的模式:

Country (*Name*: varchar(35),
Cont: varchar(35),
Pop: Integer,
GDP: Numeric,
Life-Exp: Numeric,
Shape: Polygonid)

City (*Name*: varchar(35),
Country: varchar(35),
Pop: Integer,
Capital: char(1) ,
Shape: Pointid)

River (*Name*: varchar(35),
Origin-Country: varchar(35),
Length: Numeric,
Shape: Lineid)

扩展的SQL查询语言

OGIS的SQL空间扩展

Country表

Name	Cont	Pop (m)	GDP (b)	Life-Exp	Shape
Canada	NAM	30.1	658.0	77.08	Polygonid-1
Mexico	NAM	107.5	694.3	69.36	Polygonid-2
Brazil	SAM	183.3	1004.0	65.60	Polygonid-3
Cuba	NAM	11.7	16.9	75.95	Polygonid-4
USA	NAM	270.0	8003.0	75.75	Polygonid-5
Argentina	SAM	36.3	348.2	70.75	Polygonid-6

City表

Name	Country	Pop (m)	Capital	Shape
Havana	Cuba	2.1	Y	Pointid-1
Washington, D.C	USA	3.2	Y	Pointid-2
Monterrey	Mexico	2.0	N	Pointid-3
Toronto	Canada	3.4	N	Pointid-4
Brasilia	Brazil	1.5	Y	Pointid-5
Ottawa	Canada	0.8	Y	Pointid-6

River表

Name	Origin	Length (km)	Shape
Rio Parana	Brazil	2600	LineStringid-1
St. Lawrence	USA	1200	LineStringid-2
Rio Grande	USA	3000	LineStringid-3
Mississippi	USA	6000	LineStringid-4

■ 扩展的SQL查询语言

OGIS的SQL空间扩展

查询1：列出Country表中所有与美国相邻的国家

```
SELECT      C1.Name AS “Neighbors of USA ”  
FROM        Country C1, Country C2  
WHERE       Touch (C1.Shape, C2.Shape) = 1  
            AND   C2.Name=“USA”
```

谓词Touch 检测两个几何对象是否彼此相邻而又不交叠。OGIS中定义的8个拓扑谓词之一。

■ 扩展的SQL查询语言

OGIS的SQL空间扩展

查询2：找出河流流经的国家

```
SELECT      R.Name C.Name  
FROM        River R, Country C  
WHERE       Cross (R.Shape, C.Shape)
```

拓扑谓词Cross常用于线对象与多边形对象的穿过判断。

扩展的SQL查询语言

OGIS的SQL空间扩展

查询3：对于River表中列出的所有河流，找出距离河流最近的城市。

```
SELECT      C1.Name R1.Name
FROM        City C1, River R1
WHERE       Distance (C1.Shape, R1.Shape) <
            ALL (
                SELECT Distance (C2.Shape, R1.Shape)
                FROM City C2
                WHERE  C1.Name<>C2.Name )
```

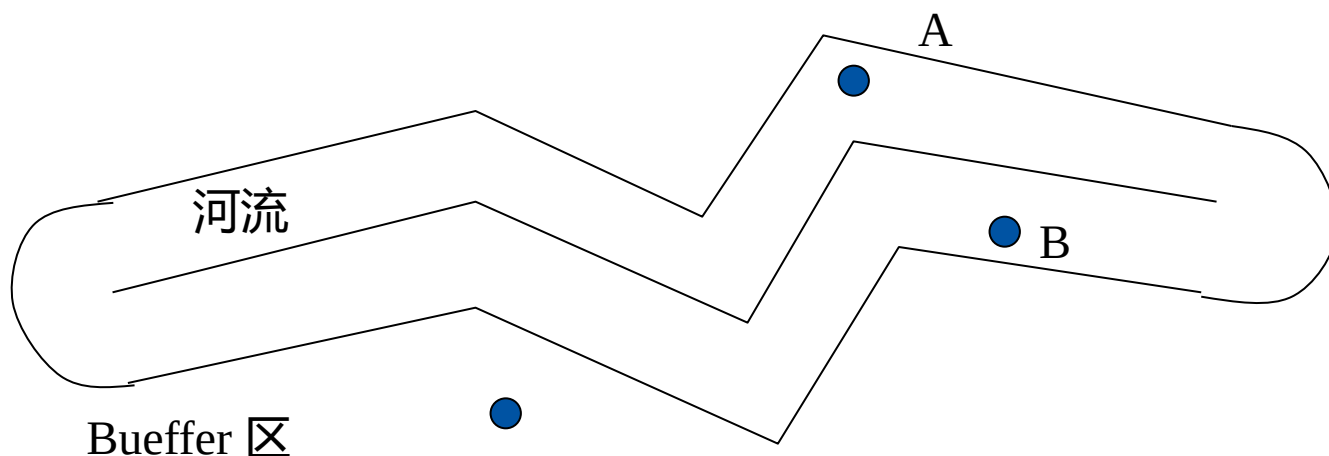
Distance 一个 二元运算，返回实数值。

扩展的SQL查询语言

OGIS的SQL空间扩展

查询4：圣劳伦斯河能为方圆300公里以内的城市供水，列出能从该河获得供水的城市。

```
SELECT    Ci.Name
FROM      City Ci, River R
WHERE     Overlap(Ci.Shape, Buffer(R.Shape, 300))=1 AND
          R.Name = "St.Lawrence"
```



■ 扩展的SQL查询语言

OGIS的SQL空间扩展

查询5：列出Country表中每个国家的名字、人口和国土面积

```
SELECT      C.Name, C.Pop, Area(C.Shape) AS “Area”  
FROM        Country C
```

面积函数 Area, 只适用于Polygon 和 MultiPolygon

■ 扩展的SQL查询语言

OGIS的SQL空间扩展

查询6： 求出河流在流经的各国境内的长度

```
SELECT      R.Name, C.Name, Length (Intersection (R.Shape, C.Shape))  
            AS “Length”  
  
FROM        River R, Country C  
  
WHERE       Cross (R.Shape, C.Shape ) =1
```

Intersection () 函数计算两个几何对象的交部分。
Intersects 是一个拓扑谓词。

扩展的SQL查询语言

OGIS的SQL空间扩展

查询7： 列出每个国家的GDP以及其首都到赤道的距离。

```
SELECT      Co.GDP, Distance ( Point( 0, Ci.Shape.x), Ci.Shape)
            AS "Distance"
FROM        Country Co, City Ci
WHERE       Co.Name=Ci.Country AND
            Ci.Captial = "Y"
```

联机分析处理 OLAP, 数据挖掘

■ 扩展的SQL查询语言

OGIS的SQL空间扩展

查询8：按其邻国数目的多少列出所有国家

```
SELECT      Co.Name, Count (Co1.Name)
FROM        Country Co, Country Co1
WHERE       Touch ( Co.Shape, Co1.Shape )
            GROUP BY Co.Name
            GROUP BY Count(Co1.Name)
```

■ 扩展的SQL查询语言

OGIS的SQL空间扩展

查询9：哪一个国家的邻国最多

```
CREATE    VIEW  Neighbor  AS
SELECT    Co.Name, Count (Co1.Name) AS num-neighbors
FROM      Country Co, Country Co1
WHERE     Touch ( Co.Shape, Co1.Shape )
GROUP BY  Co.Name

SELECT    Co.Name, num_neighbors
FROM      Neighbor
WHERE     num_neighbor = (SELECT Max (num_neighbors)
                          FROM Neighbor )
```

■ 扩展的SQL查询语言

OGIS的SQL空间扩展

- **OGIS标准的局限性：**
 - 1) 支持空间对象模型，对场模型支持不好。
 - 2) 对象模型查询操作存在局限性。（GROUP BY , HAVING存在问题）
 - 3) 不支持基于方位的谓词操作（比如，东南西北，上下左右前后等）
 - 4) 不支持基于形状、基于可见性等的操作。

■ 扩展的SQL查询语言

对象关系SQL

SQL3概览

- SQL3/SQL99主要在以下两方面进行扩展：
- 1) ADT：可以使用CREATE TYPE语句来定义ADT。ADT由一组属性和访问这些属性值的成员函数组成，成员函数可以隐含地修改数据类型中的属性值，也就能够改变数据库的状态。例如用ADT定义列属性的类型：

```
CREATE TYPE Point (  
  x NUMBER,  
  y NUMBER  
  FUNCTION Distance (: u Point, : v Point)  
  RETURNS NUMBER)
```

- u和v前的冒号表示这两个变量是局部变量。

扩展的SQL查询语言

对象关系SQL

SQL3概览

- 2) 行类型：行类型是用于定义关系的类型，它指定了关系的模式。
例如：

```
CREATE ROW TYPE Point (  
  x NUMBER,  
  y NUMBER) ;
```

可以创建一个表作为这个行类型的实例：

```
CREATE TABLE PointTable of TYPE Point;
```

注意：用ADT作为列类型能很自然地与OR-DBMS保持一致

2.2 扩展的SQL查询语言

对象关系SQL

- 建立点对象

```
CREATE TYPE Point AS OBJECT (  
    x NUMBER,  
    y NUMBER,  
    MEMBER FUNCTION Distance ( P2 IN Point ) RETURN NUMBER  
    PRAGMA RESTRICT_REFERENCES ( Distance, WNDS));  
.....
```

- City的对象-关系模式可定义为

```
CREATE TABLE City (  
    Name varchar(30)  
    Country varchar(30)  
    Pop Number  
    Shape Point );
```

2.2 扩展的SQL查询语言

对象关系SQL

创建河流的例子

```
CREATE TYPE River (  
    Name Varchar (30)  
    Origin Varchar(30)  
    Length number,  
    Shape LineString );
```

```
INSERT INTO RIVER (“MISSISSIPPI”, “USA”, 6000, LineString(3,  
LineType(Point(1,1), Point(1,2), Point(2,3)))
```

2.2 扩展的SQL查询语言

对象关系SQL

利用Polygon 建立国家表格

```
CREATE TABLE Country (
```

```
    Name    Varchar (30)
```

```
    Cont    Varchar (30)
```

```
    Pop     int,
```

```
    GDP     number,
```

```
    Life-Exp number,
```

```
    Shape   Polygon );
```

```
INSERT INTO Country ( "Mexico", "NAM", 107.5, 694.3, 69.36, Polygon(23,  
Polytype(Point(1,1) ,.....,Point(1,1)))
```

2.2 扩展的SQL查询语言

对象关系SQL

查询示例

找出City表列出的所有城市两两之间的距离

```
SELECT  C1.Name, C1.Distance(C2.Shape) AS "Distance"  
FROM    City C1, City C2  
WHERE   C1.Name <> C2.Name
```

列出与美国接壤的国家的名字、人口和国土面积

```
SELECT  C2.Name, C2.Pop, C2.Area() AS "Area"  
FROM    Country C1, Country C2  
WHERE   C1.Name = "USA" AND  
        C1.Touch(C2.Shape) = 1
```

ADT函数

拓扑谓词

3.1 PostGIS空间函数

1. OGC标准函数

管理函数:

添加几何字段 `AddGeometryColumn(, , , , )`

删除几何字段 `DropGeometryColumn(, , )`

检查数据库几何字段并在geometry_columns中归档 `Probe_Geometry_Columns()`

给几何对象设置空间参考（在通过一个范围做空间查询时常用） `ST_SetSRID(geometry, integer)`

3.1 PostGIS空间函数

1. OGC标准函数

几何对象关系函数：

获取两个几何对象间的距离 ST_Distance(geometry, geometry)

如果两个几何对象间距离在给定值范围内，则返回TRUE ST_DWithin(geometry, geometry, float)

判断两个几何对象是否相等

(比如LINESTRING(0 0, 2 2)和LINESTRING(0 0, 1 1, 2 2)是相同的几何对象) ST_Equals(geometry, geometry)

判断两个几何对象是否分离 ST_Disjoint(geometry, geometry)

判断两个几何对象是否相交 ST_Intersects(geometry, geometry)

判断两个几何对象的边缘是否接触 ST_Touches(geometry, geometry)

判断两个几何对象是否互相穿过 ST_Crosses(geometry, geometry)

判断A是否被B包含 ST_Within(geometry A, geometry B)

判断两个几何对象是否是重叠 ST_Overlaps(geometry, geometry)

判断A是否包含B ST_Contains(geometry A, geometry B)

判断A是否覆盖 B ST_Covers(geometry A, geometry B)

判断A是否被B所覆盖 ST_CoveredBy(geometry A, geometry B)

通过DE-9IM 矩阵判断两个几何对象的关系是否成立 ST_Relate(geometry, geometry, intersectionPatternMatrix)

获得两个几何对象的关系 (DE-9IM矩阵) ST_Relate(geometry, geometry)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象处理函数:

获取几何对象的中心 ST_Centroid(geometry)

面积量测 ST_Area(geometry)

长度量测 ST_Length(geometry)

返回曲面上的一个点 ST_PointOnSurface(geometry)

获取边界 ST_Boundary(geometry)

获取缓冲后的几何对象 ST_Buffer(geometry, double, [integer])

获取多几何对象的外接对象 ST_ConvexHull(geometry)

获取两个几何对象相交的部分 ST_Intersection(geometry, geometry)

将经度小于0的值加360使所有经度值在0-360间 ST_Shift_Longitude(geometry)

获取两个几何对象不相交的部分 (A、B可互换) ST_SymDifference(geometry A, geometry B)

从A去除和B相交的部分后返回 ST_Difference(geometry A, geometry B)

返回两个几何对象的合并结果 ST_Union(geometry, geometry)

返回一系列几何对象的合并结果 ST_Union(geometry set)

用较少的内存和较长的时间完成合并操作, 结果和ST_Union相同 ST_MemUnion(geometry set)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象存取函数:

获取几何对象的WKT描述 ST_AsText(geometry)

获取几何对象的WKB描述 ST_AsBinary(geometry)

获取几何对象的空间参考ID ST_SRID(geometry)

获取几何对象的维数 ST_Dimension(geometry)

获取几何对象的边界范围 ST_Envelope(geometry)

判断几何对象是否为空 ST_IsEmpty(geometry)

判断几何对象是否不包含特殊点（比如自相交） ST_IsSimple(geometry)

判断几何对象是否闭合 ST_IsClosed(geometry)

判断曲线是否闭合并且不包含特殊点 ST_IsRing(geometry)

获取多几何对象中的对象个数 ST_NumGeometries(geometry)

获取多几何对象中第N个对象 ST_GeometryN(geometry,int)

获取几何对象中的点个数 ST_NumPoints(geometry)

获取几何对象的第N个点 ST_PointN(geometry,integer)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象存取函数:

获取多边形的外边缘 ST_ExteriorRing(geometry)

获取多边形内边界个数 ST_NumInteriorRings(geometry)

同上 ST_NumInteriorRing(geometry)

获取多边形的第N个内边界 ST_InteriorRingN(geometry,integer)

获取线的终点 ST_EndPoint(geometry)

获取线的起始点 ST_StartPoint(geometry)

获取几何对象的类型 GeometryType(geometry)

类似上, 但是不检查M值, 即POINTM对象会被判断为point

ST_GeometryType(geometry)

获取点的X坐标 ST_X(geometry)

获取点的Y坐标 ST_Y(geometry)

获取点的Z坐标 ST_Z(geometry)

获取点的M值 ST_M(geometry)

3.1 PostGIS空间函数

1. OGC标准函数

几何对象构造函数：

参考语义：

Text: WKT

WKB: WKB

Geom: Geometry

M: Multi

Bd: BuildArea

Coll: Collection ST_GeomFromText(text,[])

ST_PointFromText(text,[])
ST_LineFromText(text,[])
ST_LineStringFromText(text,[])
ST_PolyFromText(text,[])
ST_PolygonFromText(text,[])
ST_MPointFromText(text,[])
ST_MLineFromText(text,[])
ST_MPolyFromText(text,[])
ST_GeomCollFromText(text,[])
ST_GeomFromWKB(bytea,[])
ST_GeometryFromWKB(bytea,[])
ST_PointFromWKB(bytea,[])
ST_LineFromWKB(bytea,[])
ST_LineStringFromWKB(bytea,[])
ST_PolyFromWKB(bytea,[])
ST_PolygonFromWKB(bytea,[])
ST_MPointFromWKB(bytea,[])
ST_MLineFromWKB(bytea,[])
ST_MPolyFromWKB(bytea,[])
ST_GeomCollFromWKB(bytea,[])
ST_BdPolyFromText(text WKT, integer SRID)
ST_BdMPolyFromText(text WKT, integer SRID)

3.1 PostGIS空间函数

2. PostGIS扩展函数

管理函数：

删除一个空间表（包括geometry_columns中的记录）

DropGeometryTable([],)

更新空间表的空间参考 UpdateGeometrySRID([], , ,)

更新空间表的统计信息 update_geometry_stats([,])

参考语义：

Geos：GEOS库

Jts：JTS库

Proj：PROJ4库 postgis_version()

postgis_lib_version()

postgis_lib_build_date()

postgis_script_build_date()

postgis_scripts_installed()

postgis_scripts_released()

postgis_geos_version()

postgis_jts_version()

postgis_proj_version()

postgis_uses_stats()

postgis_full_version()

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何量测函数:

量测面积 ST_Area(geometry)

根据经纬度点计算在地球表面上的距离, 单位米, 地球半径取值6370986米 ST_distance_sphere(point, point)

类似上, 使用指定的地球椭球参数 ST_distance_spheroid(point, point, spheroid)

量测2D对象长度 ST_length2d(geometry)

量测3D对象长度 ST_length3d(geometry)

根据经纬度对象计算在地球表面上的长度 ST_length_spheroid(geometry,spheroid)

ST_length3d_spheroid(geometry,spheroid)

量测两个对象间距离 ST_distance(geometry, geometry)

量测两条线之间的最大距离 ST_max_distance(linestring,linestring)

量测2D对象的周长 ST_perimeter(geometry)

ST_perimeter2d(geometry)

量测3D对象的周长 ST_perimeter3d(geometry)

量测两点构成的方位角, 单位弧度 ST_azimuth(geometry, geometry)

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何对象输出:

参考语义:

NDR: Little Endian

XDR: big-endian

HEXEWKB: Canonical

SVG: SVG 格式

GML: GML 格式

KML: KML 格式

GeoJson: GeoJson 格式

`ST_AsBinary(geometry, { 'NDR' | 'XDR' })`

`ST_AsEWKT(geometry)`

`ST_AsEWKB(geometry, { 'NDR' | 'XDR' })`

`ST_AsHEXEWKB(geometry, { 'NDR' | 'XDR' })`

`ST_AsSVG(geometry, [rel], [precision])`

`ST_AsGML([version], geometry, [precision])`

`ST_AsKML([version], geometry, [precision])`

`ST_AsGeoJson([version], geometry, [precision], [options])`

3.1 PostGIS空间函数

2. PostGIS扩展函数

几何对象创建:

参考语义:

Dump: 转储 ST_GeomFromEWKT(text)

ST_GeomFromEWKB(bytea)
ST_MakePoint(, , [], [])
ST_MakePointM(, ,)
ST_MakeBox2D(,)
ST_MakeBox3D(,)
ST_MakeLine(geometry set)
ST_MakeLine(geometry, geometry)
ST_LineFromMultiPoint(multipoint)
ST_MakePolygon(linestring, [linestring[]])
ST_BuildArea(geometry)
ST_Polygonize(geometry set)
ST_Collect(geometry set)
ST_Collect(geometry, geometry)
ST_Dump(geometry)
ST_DumpRings(geometry)

3.2 PostGIS空间查询语言

nyc_census_blocks **人口普查块** (**census block**) 是人口普查数据的最小地理单位。

blkid	A 15-digit code that uniquely identifies every census block . Eg: 360050001009000
popn_total	Total number of people in the census block
popn_white	Number of people self-identifying as "White" in the block
popn_black	Number of people self-identifying as "Black" in the block
popn_nativ	Number of people self-identifying as "Native American" in the block
popn_asian	Number of people self-identifying as "Asian" in the block
popn_other	Number of people self-identifying with other categories in the block
boroname	Name of the New York borough. Manhattan, The Bronx, Brooklyn, Staten Island, Queens
geom	Polygon boundary of the block https://blog.csdn.net/qq_35732147

nyc_neighborhoods **社区**

name	Name of the neighborhood
boroname	Name of the New York borough. Manhattan, The Bronx, Brooklyn, Staten Island, Queens
geom	Polygon boundary of the neighborhood

nyc_streets **街道线**

name	Name of the street
oneway	Is the street one-way? "yes" = yes, "" = no
type	Road type (primary, secondary, residential, motorway)
geom	Linear centerline of the street

nyc_subway_stations **地铁站**

name	Name of the station
borough	Name of the New York borough. Manhattan, The Bronx, Brooklyn, Staten Island, Queens
routes	Subway lines that run through this station
transfers	Lines you can transfer to via this station
express	Stations where express trains stop, "express" = yes, "" = no
geom	Point location of the station

nyc_census_sociodata **人口普查区域对应的社会人口经济数据**

tractid	An 11-digit code that uniquely identifies every census tract . ("36005000100")
transit_total	Number of workers in the tract
transit_private	Number of workers in the tract who use private automobiles / motorcycles
transit_public	Number of workers in the tract who take public transit
transit_walk	Number of workers in the tract who walk
transit_other	Number of workers in the tract who use other forms like walking / biking
transit_none	Number of workers in the tract who work from home
transit_time_mins	Total number of minutes spent in transit by all workers in the tract (minutes)
family_count	Number of families in the tract
family_income_median	Median family income in the tract (dollars)
family_income_mean	Average family income in the tract (dollars)
family_income_aggregate	Total income of all families in the tract (dollars)
edu_total	Number of people with educational history
edu_no_highschool_dipl	Number of people with no high school diploma
edu_highschool_dipl	Number of people with high school diploma and no further education
edu_college_dipl	Number of people with college diploma and no further education
edu_graduate_dipl	Number of people with graduate school diploma https://blog.csdn.net/c

3.2 PostGIS空间查询语言

"查看布鲁克林所有社区的名字"

```
SELECT name  
FROM nyc_neighborhoods  
WHERE boroname = 'Brooklyn';
```

'West Village'社区 (neighborhood) 的面积是多少?

```
SELECT ST_Area(geom)  
FROM nyc_neighborhoods  
WHERE name = 'West Village';
```

"小意大利 (Little Italy) 社区"有什么地铁站? 它在哪些地铁线路上?

```
SELECT s.name, s.routes  
FROM nyc_subway_stations AS s  
JOIN nyc_neighborhoods AS n  
ON ST_Contains(n.geom, s.geom)  
WHERE n.name = 'Little Italy';
```

谢谢大家!

